
Algorithm 3 PPO-Clip

Require: pretrained model $\hat{p}_{\mathbf{W}}$, subset of edges $\hat{E}$

- 1: set $\mathbf{W}^{(0)} = \mathbf{W}$, $T_{\text{PPO}} = \Theta(\log \varepsilon_{\max}/\varepsilon)$, $\alpha = \Theta(KM)$
- 2: advantage function $\hat{A}(x, y) = 1_{\{(x, y) \in \hat{E}\}}$
- 3: **for** $t = 1, \dots, T_{\text{PPO}}$ **do**
- 4: $\mathbf{W}^{(t)} = \mathbf{W}^{(t-1)} + \alpha \text{sgn}(\nabla L_{\text{PPO}}(\mathbf{W}^{(t-1)}; \hat{A}))$
- 5: **end for**

from Theorem 3.2 that the expected hitting time is improved by the factor $\varepsilon_{\max}/\varepsilon$. At the same time, the *magnitude* (total variation) of change to the pretrained model is negligible:

$$\sup_{x \in S} \|p^\varepsilon(\cdot|x) - p^{\varepsilon_{\max}}(\cdot|x)\|_{\text{TV}} \leq o(1/M),$$

so the original capabilities of the base model are generally preserved. Hence RL is also extremely efficient for fine-tuning the pretrained model to improve CoT.

4 Distillation to a Smaller Model

A prominent innovation in the LLM development pipeline is to distill CoT of a powerful model into a smaller, more efficient model. This approach has been shown to significantly enhance reasoning ability, especially compared to directly training the smaller model with RL (Shridhar et al., 2022; Hsieh et al., 2023; Gandhi et al., 2024; Guo et al., 2025). In this section, we showcase an explicit distillation scheme for our CoT model that efficiently generates the hard reasoning steps to solve any task while faithfully capturing the metastable dynamics of the original system.

4.1 Distilling Cluster Transitions

The metastable chain $q_\star^\varepsilon$ (Section 2) provides a natural notion of compression for the nearly reducible system X^ε by collapsing each cluster into a single state. For many downstream tasks (including the logic task studied in Section 5) it may be satisfactory to retrieve only the hard reasoning steps connecting the clusters containing $X_{\text{in}}, X_{\text{out}}$. In particular, if the goal is to extract only the *connectivity* of $S_\star$, it suffices to take the sparse edge estimate $\mathcal{M}_s$ of Algorithm 2 and perform a uniform random walk to find a path between any two clusters. However, we want the distilled model to also preserve the underlying dynamics of the original chain as best as possible. To this end, we implement the following process, detailed in Algorithm 4 in the appendix.

We first choose a set $S_o = \{x_1, \dots, x_K\}$ of representatives x_k of C_k and assign each state to its representative via the map $\iota : S \rightarrow S_o$; this can be done by exploring each cluster similarly to the first T_0 steps of the search process.

Data collection. The data for distillation is collected by continually running CoT and recording the frequency of transitions (or non-transitions) between $S_\circ$. This yields one datum per CoT step, and can also be implemented in parallel for an arbitrary number of independent chains.

- (1) If $X_t^\varepsilon \in S_\star$ and the previous return to $S_\circ$ was $X_{t_{\text{prev}}}^\varepsilon$ then add $(X_{t_{\text{prev}}}^\varepsilon, X_t^\varepsilon)$ to D_{dist} .
- (2) If $X_t^\varepsilon \notin S_\circ$ (no transition) add $(\iota(X_t^\varepsilon), \iota(X_t^\varepsilon))$ to D_{dist} .

This requires only $O(K^2)$ memory for frequency counts; no cache for X^ε is needed. The cluster labels ι and parameters $\mathbf{Z}$ require $O(KM)$ and $O(K^2)$ memory, respectively. We suppose the process is run for arbitrarily long time so that we have access to the population distribution of D_{dist} . We then one-hot embed $S_\circ$ in $\mathbb{R}^K$ and use the collected data pairs to train a softmax model $\hat{q}_{\mathbf{Z}}(\cdot|x) = \text{softmax}(\langle \mathbf{Z}, x \rangle)$, $\mathbf{Z} \in \mathbb{R}^{K \times K}$ similarly to pretraining. Finally, we rescale time so that the non-diagonal entries sum to $\Theta(1)$, reducing redundant within-cluster transitions.

Equivalence with meta-chain. The data $(Y_0, Y_1) \sim D_{\text{dist}}$ has been constructed so that the distilled model learns the following kernel $q_\circ^\varepsilon$ on $S_\circ$: $Y_0 \sim \pi^\varepsilon$, $Y_1 \sim q_\circ^\varepsilon(\cdot|Y_0)$ where

$$\begin{aligned} q_\circ^\varepsilon(x_\ell|x_k) &:= \pi_k^\varepsilon(x_k) \mathbb{P}_{x_k}(X_{\bar{\tau}_{S_\circ}^\varepsilon}^\varepsilon = x_\ell), \quad k \neq \ell, \\ q_\circ^\varepsilon(x_k|x_k) &:= 1 - \sum_{\ell \neq k} q_\circ^\varepsilon(x_\ell|x_k). \end{aligned}$$

This kernel is a lazy version of the process obtained from X^ε by deleting all transitions to states outside $S_\circ$, with an additional time rescaling according to the stationary probability $\pi_k^\varepsilon(x_k)$. This is slightly different from the construction given in [Betz and Le Roux \(2016\)](#), as we do not presume access to the stationary distribution of the unperturbed chain p_k^0 and must sample directly from π^ε . Moreover, $q_\circ^\varepsilon$ is dependent on the choice of representatives and thus different from the ‘canonical’ meta-chain $q_\star^\varepsilon$ in general. Nonetheless, $q_\circ^\varepsilon$ is faithful to the meta-chain in a rigorous sense:

Proposition 4.1. *Denote the return time of $q_\circ^\varepsilon$ to x_k as $\bar{\tau}_{\circ, x_k}^\varepsilon$. For all $k, \ell \in [K]$ with $k \neq \ell$, it holds that*

$$\frac{\mathbb{P}_{x_k}(\bar{\tau}_{\circ, x_\ell}^\varepsilon < \bar{\tau}_{\circ, x_k}^\varepsilon)}{q_\star^\varepsilon(C_\ell|C_k)} = 1 + o_M(1).$$

That is, the escape probabilities of $q_\circ^\varepsilon$ converge to $q_\star^\varepsilon$ with uniformly vanishing relative error. This property is desirable as it shows the eventual likelihood of escaping to each cluster (i.e., reaching a certain idea) is consistent across different choices of $S_\circ$.

4.2 CoT of Distilled Model

To analyze the utility of the trained model $\hat{p}_{\mathbf{Z}+}$, we make the additional assumption:

Assumption 5 (inbound sparse edges). *All sparse edges leading to each cluster C_k terminate at a fixed point x_k . For any sparse edge (x', x_ℓ) from C_k , there exists a path from x_k to x' in C_k of probability bounded below.*

Then we may specify $S_\circ$ as the set of the points x_k . This ensures that representatives will not be skipped; otherwise, a CoT passing through C_k, C_ℓ, C_n in succession may miss x_ℓ and record the wrong transition (x_k, x_n) (although the likelihood of this is $o(1)$ regardless).

Now, as with pretraining, the distilled model will converge to $q_\circ^\varepsilon$ when trained with cross-entropy loss on D_{dist} .

Proposition 4.2 (convergence of distillation). *For the gradient descent iterates $\mathbf{Z}^{(t)}$ from Algorithm 4, the learned probabilities converge to $q_\circ^\varepsilon$ after $T_{\text{dist}} = \tilde{O}(M^2\varepsilon^{-2})$ as*

$$\sup_{k,\ell} |\hat{q}_{\mathbf{Z}^{(T_{\text{dist}})}}(x_\ell|x_k) - q_\circ^\varepsilon(x_\ell|x_k)| = K^{-\omega(1)}.$$

We point out the time to convergence is much faster than pretraining time $\tilde{O}(KM^2\varepsilon^{-2})$ (Theorem 3.1), and also more computationally efficient since we are training a size K^2 model rather than size $(KM)^2$.

Finally, after time rescaling, the model $q_{\mathbf{Z}^+}$ is capable of efficiently finding a path from the cluster containing X_{in} to the cluster containing X_{out} , with hitting time linear in $|S_\circ|$ and independent of the difficulty parameter ε .

Theorem 4.3 (hitting time of distilled CoT). *For all $k \neq \ell$, $\hat{q}_{\mathbf{Z}^+}(x_\ell|x_k) = \Theta(1)$ if there exists a sparse edge from C_k to C_ℓ or 0 if not. Moreover, the hitting time $\tau_{x_\ell}^+$ of $x_\ell \in S_\circ$ by $\hat{q}_{\mathbf{Z}^+}$ satisfies $\mathbb{E}_{x_k}[\tau_{x_\ell}^+] = O(K)$.*

The returned sequence of clusters $C_{0:T}$ indicate the existence of a path from X_{in} to X_{out} passing through precisely these clusters in order. Once $C_{0:T}$ is determined, a weaker reasoning agent (e.g., the base model p^ε) may also efficiently resolve the fine-grained dynamics within each cluster.

5 Logical Reasoning is Hard without Search

5.1 Logical Reasoning Task

In this section, we further investigate the benefits of search for reasoning by adding a quantitative ‘logic task’ on top of the path-finding task. This provides two benefits. First, having a numerical answer allows us to evaluate the hardness of the task from a learning-theoretic perspective, separate from the previously obtained hitting time bounds. Second, by having the answer depend only on the sparse edges along a path, the reasoner is required to estimate which edges are sparse – in other words, understand which reasoning steps are actually important – in order to solve the task. Taking a proof problem for example, we expect an LLM with strong reasoning capability to not only *generate* a plausible solution via next-token prediction but also *understand* its own proof, so that it can correctly answer logical questions such as “what are the key ideas of this proof?” or “what happens if we replace step X with Y?” We attempt to formalize this notion using group actions (Definition E.1).